Знакомство с .NET.
Синтаксис, типы данных и ООП

Юрий Уфимцев

План пары

- Организационное вступление
- Теория
  - О языке C# и платформе .NET
  - О его синтаксисе
  - О его типах
  - Обсудим модификаторы видимости, абстрактные классы и интерфейсы, virtual, new и override
  - Вкратце обсудим управление ресурсами: using и IDisposable
- Практика
  - Поставим IDE и .NET
  - Посмотрим на особенности C# на практике





Вступление

О C#

- Комфортный ООП-язык
  - Как Java, но удобнее
  - Как Kotlin, но на .NET
- Востребован
  - Контур
  - Озон
  - Т-Банк
- Актуален на матмехе
  - HwProj пишется на C#
  - Есть много полезных материалов



О себе

- Бэкенд-разработчик в Т-Банке
- Несколько лет в разработке бэкенда
- СПбГУ, ИТМО
- Связаться
  - tg: @yu_ufimtsev
  - YuriUfimtsev@yandex.ru



О курсе

- Обсудим основы C# и промышленной разработки
  - Современные подходы и технологии **на примере** C#
  - Важно понимать
  - Спрашивают на собесах
- Будет несколько домашек/мини-проектов
  - Потрогать руками и закрепить знания
  - Показать на парах для контрольных
  - Сохранить для портфолио
- Основная коммуникация по курсу – HwProj





Вопросы?

Язык C#



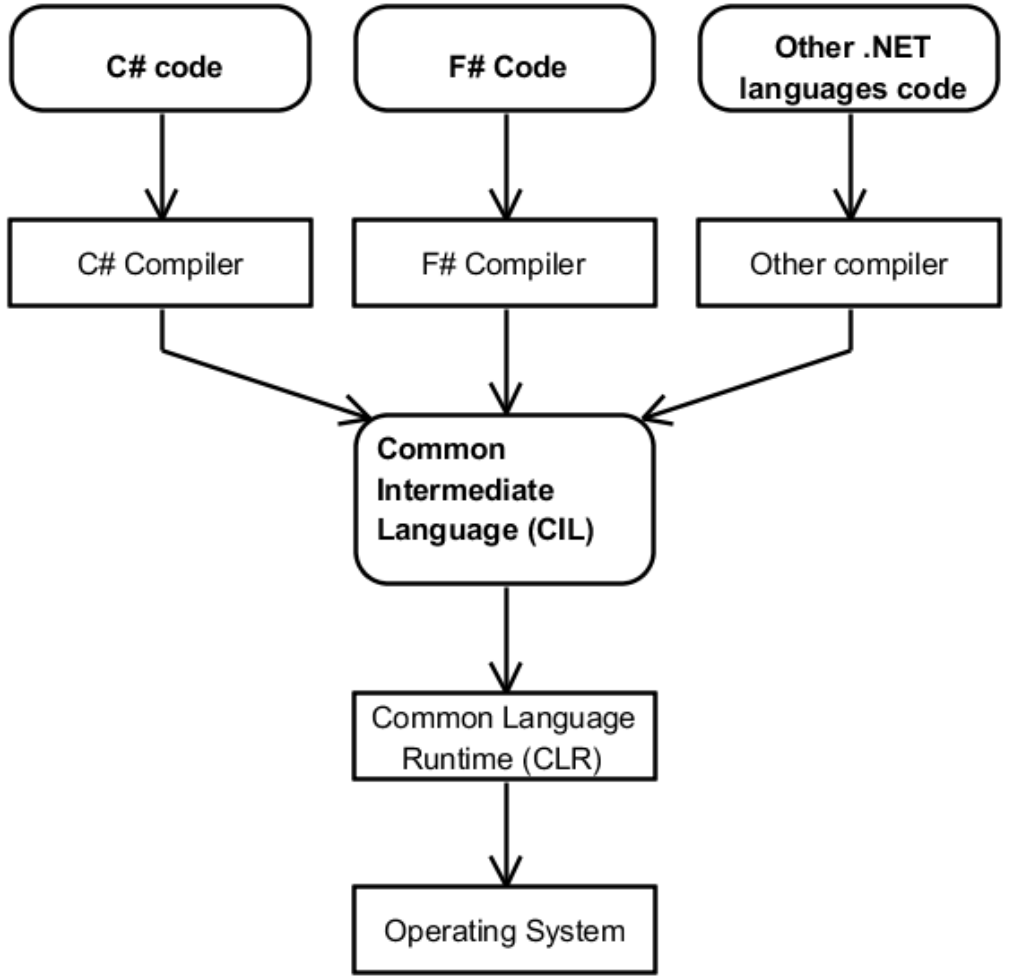
- Объектно-ориентированный язык общего назначения с сильной типизацией
- Основной язык программирования для платформы .NET
- Первая версия — 2002 год, актуальная LTS — 11.11.2025, C# 14 (.NET 10)
  - Стабильные новые версии раз в год
- 5-е место в индексе TIOBE на сентябрь 2026 (после Python, C++, Java, C)
  - https://www.tiobe.com/tiobe-index/
- Работает под Windows и Linux/Mac OS
- Средства разработки
  - Microsoft Visual Studio (https://www.visualstudio.com)
  - Visual Studio Code (https://code.visualstudio.com/) + C# DevKit
  - **Rider** (https://www.jetbrains.com/rider/)



Common Language Infrastructure



- Компиляция не в машинные коды, а в байткод виртуальной машины (Common Intermediate Language, CIL)
- Виртуальная машина и набор библиотек (Common Language Runtime) реализуется для каждой платформы (ОС), на которой хотим запускать байт-код
- Машина интерпретирует байт-код или компилирует его «на лету» в машинные коды



Зачем так



- Единый байткод с механизмом оптимизации, интерпретации и генерации машинного кода позволяет экономить время на разработку компиляторов
  - Поэтому под .NET можно писать на C#, F#, Visual Basic, куче сторонних языков (например, Delphi)
- Совместимость кода, написанного на разных языках, возможность из кода на F# без усилий использовать библиотеки на C# и наоборот
- Виртуальная машина знает всё о выполнении программы и многое может проверять (обращения к памяти, границы массивов и т.д.)
- JIT-компиляция



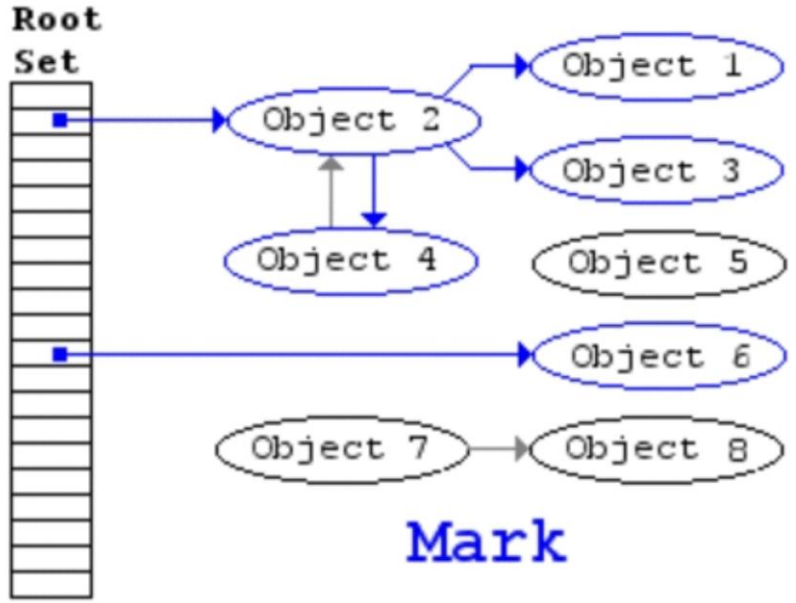
Сборка мусора



- Виртуальная машина сама следит за используемой памятью и освобождает её, когда она перестаёт быть нужной

- Корневое множество (глобальные переменные и стек вызовов), достижимое множество

- Вызывается, «когда захочет»
- Относительно вычислительно сложно
- Устроено обычно довольно хитро
  - Поколения, Large Object Heap и т.д.
  - Но разобраться нужно, и позже разберёмся



Сборка и запуск (1)



- Roslyn – компилятор C#
- MSBuild – утилита сборки

- MSBuild превращает .cs и .csproj файлы в «сборки» -- .dll или .exe файлы. «Сборки» содержат байткод виртуальной машины – тот самый промежуточный код, а также метаданные
- .NET CLR (runtime) интерпретирует байт-код или компилирует его «на лету» в машинные коды

- Существуют дизассемблеры, которые по промежуточному коду восстанавливают исходный
  - Оригинальный ildasm.exe
  - Jetbrains Disassembler в Rider

Сборка и запуск (2)



- Напрямую вызывать MSBuild для сборки уже не модно, используем утилиту **dotnet** (распространяется в составе .NET SDK)
  - **dotnet build** для сборки (под капотом MSBuild -> Roslyn)
  - **dotnet run** для запуска (уже включает в себя этап сборки)

- Скорее всего, вы будете запускаться напрямую из IDE, но знать полезно

Технические детали C#



- Hello, world:

```
System.Console.WriteLine("Goodbye, cruel world!");
```



Технические детали C#



- Или «полная форма»

```
using System;
namespace HelloWorld
{
    class Program
    {
        static void Main(string[] args)
        {
            Console.WriteLine("Goodbye, cruel world!");
        }
    }
}
```



Технические детали C#. Циклы



```
for (int i = 0; i < 300; ++i)
{
    Console.WriteLine("Мат-мех не для всех!");
}
```

- Или

```
for (var i = 0; i < 300; ++i)
{
    Console.WriteLine("Мат-мех не для всех!");
}
```



Технические детали C#. Функции



```
int Factorial(int n)
{
    if (n <= 1)
    {
        return 1;
    }
    return n * Factorial(n - 1);
}
```

- Или так

```
int Factorial(int n)
    => n <= 1 ? 1 : n * Factorial(n - 1);
```



Технические детали C#



- Использование

```
using System;

int Factorial(int n)
    => n <= 1 ? 1 : n * Factorial(n - 1);

Console.WriteLine(Factorial(4));
```



Технические детали C#



- Использование – «полная» форма

```
namespace HelloWorld
{
    class Program
    {
        private static int Factorial(int n)
        {
            return n <= 1 ? 1 : n * Factorial(n - 1);
        }
        static void Main(string[] args)
        {
            System.Console.WriteLine(Factorial(4));
        }
    }
}
```



Стайлгайд



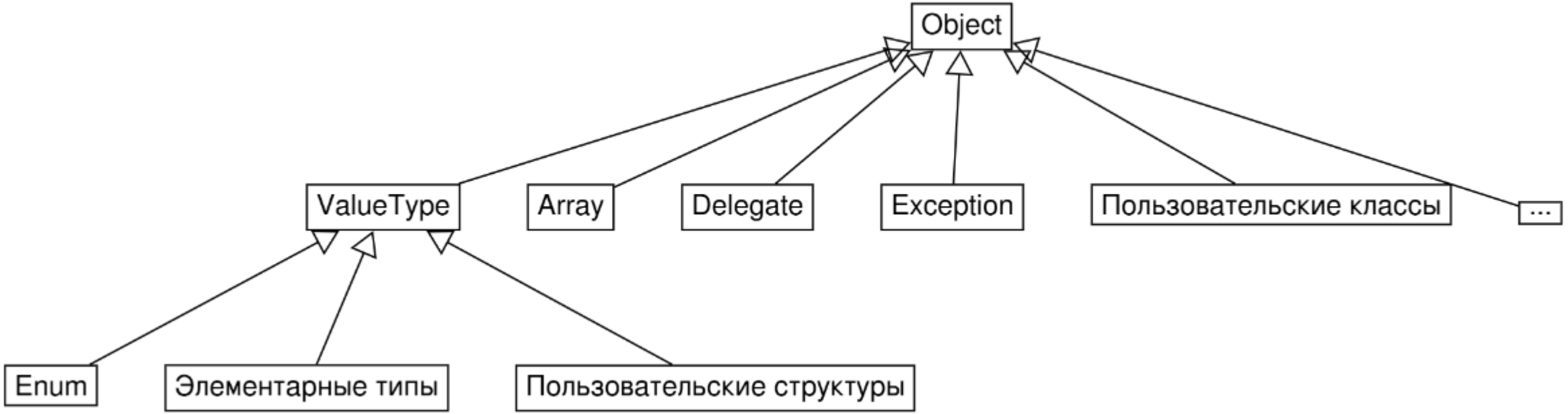
- C# Coding Conventions
  - https://learn.microsoft.com/en-us/dotnet/csharp/fundamentals/coding-style/coding-conventions
  - https://github.com/DotNetAnalyzers/StyleCopAnalyzers – полезно, но в продакшене используется мало
  - Code Analysis for Managed Code
  - **Здравый смысл**

- Позже на курсе поговорим про минимальный «чистый код»



Типы даных. Всё – объект







Методы System.Object



- **Equals** — виртуальный
- **GetHashCode** — виртуальный
- **ToString** — виртуальный
- **GetType** — невиртуальный

- MemberwiseClone — невиртуальный защищённый (просто так не вызывать)
  - Создаёт объект, не вызывая конструктор
- Finalize — виртуальный защищённый (просто так не вызывать)



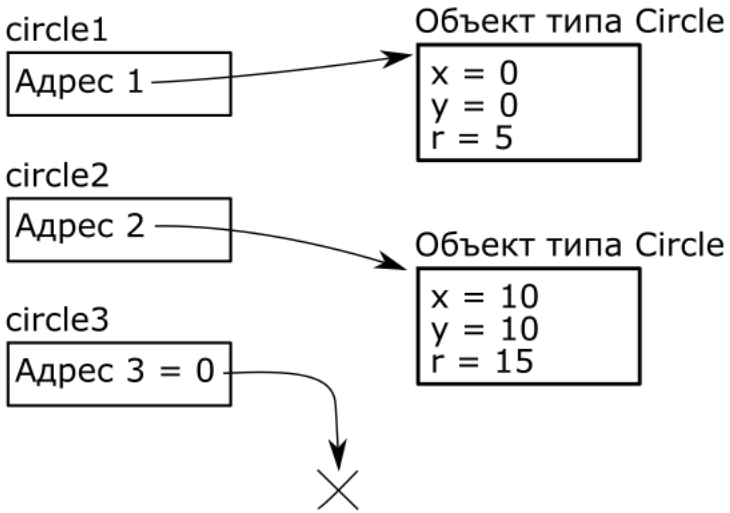
Ссылочные типы и типы-значения

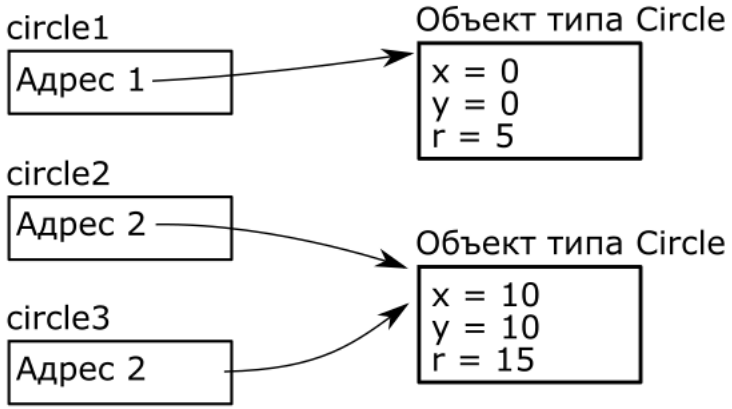


- Ссылочные типы:
  - Пользовательские классы
  - Строки
  - Массивы
  - Исключения
  - Делегаты
- Типы-значения:
  - Примитивные типы
  - Перечисления
  - Структуры



Ссылочные типы и типы-значения






```
var circle1 = new Circle(0, 0, 5);
var circle2 = new Circle(10, 10, 15);
var circle3 = null;
circle3 = circle2;
```

Примитивные типы



- Всё — объект, даже int наследуется от Object
- Типы стандартизованы: размер и множество значений одинаковы во всех реализациях
- Каждому типу соответствует библиотечный класс
  - Например, int — System.Int32
- У каждого типа есть значение по умолчанию
  - Ими поля инициализируются при создании



Методы у типов



```
var inputString = Console.ReadLine();
int number = int.Parse(inputString);
```

- Это то же самое, что

```
var inputString = Console.ReadLine();
int number = Int32.Parse(inputString);
```



Массивы



```
int[] a = new int[10];
```

или

```
var a = new int[10];
```

Пример:

```
for (var i = 0; i < a.Length; ++i)
{
    a[i] = i;
}
```

Двумерные массивы:

```
int[,] numbers = new int[3, 3];
numbers[1, 2] = 2;
int[,] numbers2 = new int[3, 3] { {2, 3, 2}, {1, 2, 6}, {2, 4, 5} };
```

Массивы знают свою длину (см. свойство Length и метод GetLength())

Структуры



```
struct Point
{
    public int x;
    public int y;
}
```

Значения копируются при присваивании



Record-ы



- Похожи на data class в Kotlin
- Неизменяемый тип данных с дополнительными возможностями
  - Equals, HashCode, ToString из коробки
  - Копирование через with, деструктуризация

```
public record Person(string FirstName, string LastName);
```

А под капотом (+конструктор, Equals, HashCode, ToString)

```
public class Person
{
    public required string FirstName { get; init; }
    public required string LastName { get; init; }
};
```



Record-ы



- То есть просто синтаксический сахар
- Бывают record struct и record class
  - По умолчанию record = record class
  - Разница ожидаемая — где хранится, как передаётся
- Record-ы могут наследоваться, в отличие от data class-ов в Kotlin



Конструкторы



```
class Circle
{
    public Circle(int x, int y, int r)
    {
        this.x = x;
        this.y = y;
        this.r = r;
    }
    private int x;
    private int y;
    private int r;
}
```



Перегрузка конструкторов, chaining



```
class Circle
{
    public Circle(int x, int y, int r)
    {
        this.x = x;
        this.y = y;
        this.r = r;
    }
    public Circle(int r)
        : this(0, 0, r)
    {
    }
    private int x;
    private int y;
    private int r;
}
```



Типы времени компиляции и времени выполнения



- Каждый объект на самом деле имеет много типов — свой и всех своих предков
- Его «настоящий» тип — тип времени выполнения
- Тип, по которому мы с ним работаем в конкретном месте программы — тип времени компиляции

```
Shape a = new Circle();
```

Тип времени компиляции — Shape, следовательно у a можно вызывать только методы Shape.
Тип времени выполнения — Circle

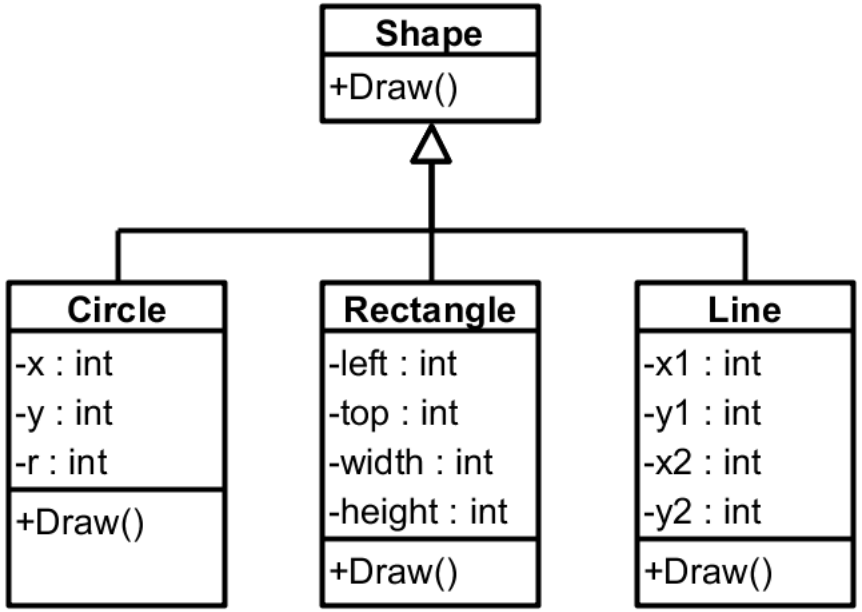


Вспомним про полиморфизм



```
List<Shape> shapes = new List
{
    new Circle(0, 0, 5),
    new Line(0, 0, 10, 10)
};

foreach (var shape in shapes)
{
    shape.Draw();
}
```





Абстрактные классы



- Иногда у предка нет разумного поведения по умолчанию
- Например, что такое Draw для просто Shape?
- Такие методы не реализуются, а остаются абстрактными
- Если у класса есть хоть один абстрактный метод, он не может порождать объекты
  - То есть сам класс является абстрактным
- **Интерфейс** — класс, у которого все методы абстрактные
  - Например, Shape мог бы быть интерфейсом
- На самом деле, интерфейс — это контракт класса



Пример



```
static void Add(string s1, string s2, string s3)
{
    s3 = s1 + s2;
}
private static void Main(string[] args)
{
    string s1 = "a";
    string s2 = "b";
    string s3 = "c";
    Add(s1, s2, s3);
}
```



Передача параметров по ссылке



```
static void Add(string s1, string s2, ref string s3)
{
    s3 = s1 + s2;
}
private static void Main(string[] args)
{
    string s1 = "a";
    string s2 = "b";
    string s3 = "c";
    Add(s1, s2, ref s3);
}
```



Out-параметры



```
static void Add(string s1, string s2, out string s3)
{
    s3 = s1 + s2;
}
private static void Main(string[] args)
{
    string s1 = "a";
    string s2 = "b";
    Add(s1, s2, out string s3);
    Console.WriteLine(s3);
}
```



На самом деле это не нужно



```
static string Add(string s1, string s2)
{
    return s1 + s2;
}
private static void Main(string[] args)
{
    string s1 = "a";
    string s2 = "b";
    string s3 = Add(s1, s2);
    Console.WriteLine(s3);
}
```

C# умеет возвращать пары, тройки и т.д.



Наследование

```
class Shape
{
    public Shape()
    {
    }
    public Shape(int x, int y)
    {
        this.x = x;
        this.y = y;
    }
    protected int x;
    protected int y;
}
```

```
class Circle : Shape
{
    Circle(int x, int y, int r)
    {
        this.x = x;
        this.y = y;
        this.r = r;
    }
    Circle(int r)
        : base(0, 0)
    {
    }
    private int r;
}
```





Интерфейсы



```
interface IDrawable
{
    void Draw();
}
```

- Принято начинать с I
  - в отличие от Java :)



Интерфейсы. Реализация



```
class Shape : IDrawable
{
    public void Draw()
    {
        Console.Write("Drawing Shape");
    }
    protected int x;
    protected int y;
}
```



Интерфейсы. Явная реализация



```
class Shape : IDrawable
{
    void IDrawable.Draw()
    {
        Console.Write("Drawing Shape");
    }
    protected int x;
    protected int y;
}
```



Интерфейсы. Явная реализация. Зачем



```
interface IDrawable {
    void Draw();
}

interface IMySpecialDrawable {
    void Draw();
}

class Shape : IDrawable, IMySpecialDrawable
{
    void IDrawable.Draw()
        => Console.Write("Drawing Shape");

    void IMySpecialDrawable.Draw()
        => Console.Write("Drawing Shape, but completely unrelated to IDrawable");

    protected int x;
    protected int y;
}
```

Интерфейсы. Явная реализация. Зачем



```
var shape = new Shape();

shape.Draw(); // Ошибка компиляции

((IDrawable)shape).Draw(); // Drawing Shape

((IMySpecialDrawable)shape).Draw(); // Drawing Shape, but...
```



Абстрактные классы



```
abstract class Shape
{
    public Shape()
    {
    }

    public abstract void Draw();

    protected int x;
    protected int y;
}
```



Виртуальные методы (1)



```
class Shape
{
    public virtual void Draw()
    {
        Console.WriteLine(
            $"Drawing Shape with coords ({x}, {y})");
    }
    protected int x;
    protected int y;
}
```



Виртуальные методы (2)



```
class Circle : Shape
{
    public Circle(int x, int y, int r)
    {
        this.x = x;
        this.y = y;
        this.r = r;
    }
    public override void Draw()
    {
        Console.WriteLine($"Drawing Circle with radius {r}");
    }
    private int r;
}
```



Виртуальные методы (3)



```
class Rectangle : Shape
{
    public Rectangle(int x, int y, int width, int height)
    {
        this.x = x;
        this.y = y;
        this.width = width;
        this.height = height;
    }
    public override void Draw()
    {
        base.Draw();
        Console.WriteLine(
            $"Drawing Rectangle with width={width} and height={height}");
    }
    protected int width;
    protected int height;
}
```

Виртуальные методы (4)



```
private static void Main(string[] args)
{
    var circle = new Circle(0, 0, 10);
    var rectangle = new Rectangle(0, 0, 10, 10);
    var list = new System.Collections.Generic.List<Shape>();
    list.Add(circle);
    list.Add(rectangle);
    foreach (var shape in list)
    {
        shape.Draw();
    }
}
```



Абстрактные методы



```
abstract class Shape
{
    public abstract void Draw();

    protected int x;
    protected int y;
}
```



Перевведение (перекрытие) методов, new (1)



```
class Shape
{
    public virtual void Draw()
    {
        Console.WriteLine(
            $"Drawing Shape with coords ({x}, {y})");
    }
    protected int x;
    protected int y;
}
```



Перевведение (перекрытие) методов, new (2)



```
class Circle : Shape
{
    public Circle(int x, int y, int r)
    {
        this.x = x;
        this.y = y;
        this.r = r;
    }
    public new void Draw()
    {
        Console.WriteLine($"Drawing Circle with radius {r}");
    }
    private int r;
}
```



Перевведение (перекрытие) методов, new (3)



```
Circle circle1 = new Circle(10, 10, 3);
Shape circle2 = new Circle(10, 10, 3);
var circle3 = new Circle(10, 10, 3);

circle1.Draw();
circle2.Draw();
circle3.Draw();
```

Лучше про это забыть и не пользоваться, но на собеседованиях спрашивают



Управление ресурсами



- Ресурсы бывают управляемые и неуправляемые
  - Память в C# – управляемый ресурс, за неё отвечает сборщик мусора (GC)
  - Файлы, соединения с БД, сетевые сокеты, дескрипторы ОС – неуправляемые (внешние) ресурсы. После того, как их использование в программе завершилось, их освобождением занимается разработчик – иначе утечка, блокирование ресурса

- Для удобного освобождения неуправляемых ресурсов в C# существует механизм **using и IDisposable**



IDIsposable



- Интерфейс IDisposable

```
public interface IDisposable
{
    void Dispose();
}
```

- В Dispose принято размещать логику освобождения ресурса
- Обычно в формате

```
try
{
    // какая-то логика работы с obj
}
finally
{
    obj.Dispose();
}
```



using



- Чтобы не писать try+finally с Dispose, придумали using
- using – просто синтаксический сахар

```
using (var obj = new SomeDisposableObj())
{
    // какая-то логика работы с obj
}
// obj.Dispose() сработает в конце блока
```

- Иногда можно так

```
using var obj = new SomeDisposableObj();
```

- В java аналог using называется try-with-resources



Модификаторы видимости



- **public** — применяется к типам и членам, доступ без ограничений
- **protected** — применяется только к членам, доступ в типе и потомках
- **internal** — применяется к типам и членам, доступ внутри сборки
- **protected internal** — применяется к типам и членам, доступ внутри сборки или в потомках
- **private** — применяется к типам и членам, доступ только внутри типа
- По умолчанию для типов **internal**, для членов — **private**



Другие модификаторы



- **partial** — «частичный» класс, декларирует, что определение класса разбито на несколько файлов
  - Для интеграции сгенерированного и рукописного кода, не используйте без нужды
- **sealed** — запрещение наследования от класса
- **static** — не может быть инстанциирован, может содержать только **static-методы** (похоже на companion object в kotlin)



Enum



В C# — константы на основе целочисленных значений (byte, int, long, …)

```
enum SomeEnum
{
    red,
    green,
    blue
}
```

то же самое, что

```
enum SomeEnum
{
    red = 0,
    green = 1,
    blue = 2
}
```

Использование:

```
SomeEnum a = SomeEnum.blue;
```

(ну или через var)

```
var a = SomeEnum.blue;
```



Вложенные классы



```
class Circle
{
    private readonly Point pos;
    private readonly int r;
    private class Point
    {
        public int x;
        public int y;
    }
    public Circle(int x, int y, int r)
    {
        pos = new Point {x = 10, y = 10};
        this.r = r;
    }
    public void Draw() =>
        Console.WriteLine($"({pos.x}, {pos.y}), radius {r}");
}
```



Преобразования типов



```
Shape shape = new Circle();
Circle circle = (Circle)shape;

Circle circle = shape as Circle;

if (shape is Circle)
{
    Circle circle = (Circle)shape;
}

if (shape is Circle circle)
{
}
```



Сопоставление шаблонов



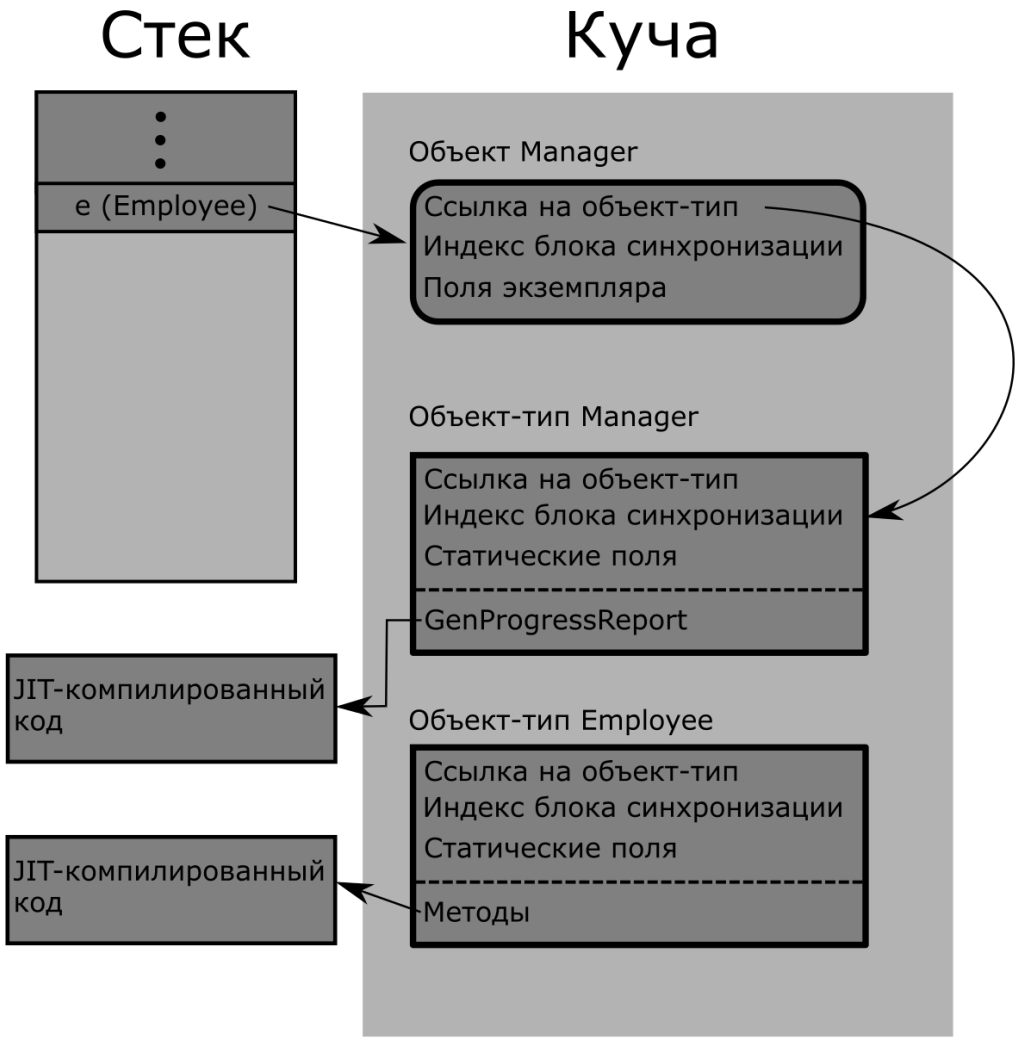
```
switch (shape)
{
    case Circle c:
        WriteLine($"circle with radius {c.Radius}");
        break;
    case Rectangle s when (s.Length == s.Height):
        WriteLine($"{s.Length} x {s.Height} square");
        break;
    case Rectangle r:
        WriteLine($"{r.Length} x {r.Height} rectangle");
        break;
    default:
        WriteLine("<unknown shape>");
        break;
    case null:
        throw new ArgumentNullException(nameof(shape));
}
```



Объекты в памяти



```
void Example()
{
    Employee e =
        new Manager();
    e.GenProgressReport();
}
```





Литература



- Конспекты Юрия Викторовича Литвинова
  - По ООП и основам C# — https://github.com/yurii-litvinov/courses/tree/master/programming-2nd-semester
  - По многопоточке, рефлексии и «углубленному .NET»: https://github.com/yurii-litvinov/courses/tree/master/programming-3rd-semester
  - Читаются легко, полезного много
- https://devblogs.microsoft.com/dotnet/ — официальный блог о .NET
- Джепикс, Троелсен Язык программирования C# 9 и платформа .NET 5 — C# для самых маленьких
- Джозеф Албахари, Бен Албахари C# 9.0. Справочник. Полное описание языка
- Jeffrey Richter CLR via C# — must read про C#, хоть и не всё уже актуально
  - Или её русское издание «CLR via C#. Программирование на платформе Microsoft.NET Framework 4.5 на языке C#»
- https://ulearn.me/ — набор открытых онлайн-курсов от «Контур» и УрГУ



Теперь практика

1. Ставим Rider: https://www.jetbrains.com/rider/download. Если проблемы с VPN, то https://drive.google.com/drive/folders/13gztiyc_jFexCcZBECzwXwCt48wesrAO?usp=sharing
2. Ставим .NET SDK 10: https://dotnet.microsoft.com/en-us/download
3. Клонируем репозиторий с демо-проектом: https://github.com/YuriUfimtsev/CSharpDemo